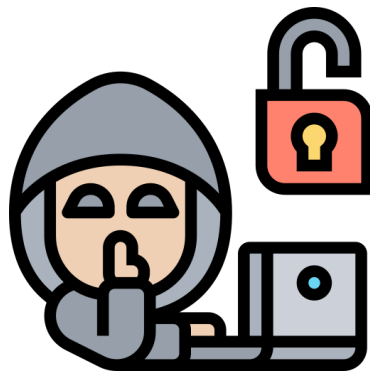


Implémentation et cryptanalyse d'un système de diffusion de données chiffrées (Broadcast Encryption)

LU2IN013

David Pulido Cornejo, Kevin Huang, Lounes Douar



Projet de recherche niveau licence encadré par Damien Vergnaud



Double Majeure Mathématiques - Informatique
Sorbonne Université
Paris, France
25 juin 2023

Table des matières

1	Introduction	2
2	Le système d’Eskeland	3
2.1	Préliminaires et configuration	3
2.2	Chiffrement	4
2.3	Déchiffrement	4
2.4	Validité	4
2.5	Sélection des paramètres	5
3	L’implémentation du système	5
3.1	Résultats expérimentaux	7
4	Les attaques	8
4.1	Le cas de plusieurs clés secrètes	9
4.2	Le cas de deux clés secrètes	9
4.2.1	Factorisation par PGCD	9
4.2.2	Fractions continues	10
5	Conclusion	12
6	Références	13
7	Annexes	14
7.1	Analyse théorique de la génération des nombres premiers	14
7.2	Exemple d’une fraction continue	15
7.3	Démonstration de l’application du théorème de Legendre	16

1 Introduction

Un cryptosystème est un schéma composé d'un ensemble d'algorithmes permettant d'effectuer trois opérations fondamentales : chiffrer un texte clair pour produire un cryptogramme ou texte chiffré, déchiffrer un texte chiffré et générer les clés nécessaires à ces deux opérations. Historiquement, les cryptosystèmes étaient basés sur des opérations rudimentaires sur l'alphabet (comme par exemple, les chiffrements de César et de Vigenère). Au fil du temps, ces opérations ont commencé à se compliquer. Aujourd'hui, les cryptosystèmes reposent principalement sur des opérations d'arithmétique modulaire et sur des propriétés de la théorie des nombres qui, combinées à la puissance de calcul des ordinateurs modernes, garantissent une sécurité maximale.

Indépendamment de la description des algorithmes de chiffrement et de déchiffrement et des propriétés arithmétiques qu'ils utilisent, ces systèmes sont classés selon la manière dont les clés sont gérées. La cryptographie asymétrique (également connue sous le nom de cryptographie à clé publique) est le domaine des cryptosystèmes qui utilise des paires de clés liées. Chaque paire de clés est constituée d'une clé publique et d'une clé privée correspondante. L'expéditeur (appelé traditionnellement Alice) utilise la clé publique du destinataire (appelé traditionnellement Bob) pour lui envoyer un message et Bob utilise sa clé privée pour le déchiffrer. L'exemple le plus populaire d'un système asymétrique est le cryptosystème RSA dont la clé publique d'un certain utilisateur est un couple d'entiers (e, n) et sa clé privée est un couple d'entiers (d, n) . Ce qui relie la clé privée et la clé publique dans ce système est le fait que $ed \equiv 1 \pmod{\phi(n)}$ où $n = pq$ avec p et q étant des nombres premiers assez grands (au moins 1024 bits) et $\phi(n) = (p-1)(q-1)$ désigne la fonction indicatrice d'Euler de n . En revanche, en cryptographie symétrique, une seule clé privée est disponible qui sera utilisée à la fois pour le chiffrement et le déchiffrement. Un exemple classique est l'AES (Advanced Encryption System) une standardisation du système de chiffrement par bloc Rijndael. Pour l'AES, la clé unique est codée sur 128, 192 ou 256 bits. Plus la taille de la clé augmente, plus notre message sera sécurisé, mais cette sécurité se fera au détriment de la performance. On voit immédiatement que compte tenu des tailles des clés, les opérations d'un système symétrique sont beaucoup plus rapides que celles d'un système asymétrique [9].

Objectif

Le cœur de notre projet s'articule autour d'un système de diffusion de données chiffrées (appelé *broadcast encryption* en anglais et que nous appellerons chiffrement broadcast dans la suite). Ce type de chiffrement permet à un expéditeur de partager en toute sécurité un message avec plusieurs destinataires à l'aide d'un seul texte chiffré, contrairement au chiffrement conventionnel où l'expéditeur doit calculer un texte chiffré unique pour chaque destinataire. Il existe plusieurs façons d'implémenter le chiffrement broadcast. Par exemple, on pourrait donner un couple clé publique et clé privée pour chaque destinataire, mais ce processus devient beaucoup trop long lorsque le nombre d'utilisateur augmente. On pourrait aussi avoir un système qui génère une clé privée unique et commune aux destinataires qui sera calculée à partir de la liste des destinataires, mais ce système pose des problèmes de collusion et d'anonymat.

Dans l'article *Collusion-resistant broadcast encryption based on hidden RSA subgroups* paru en 2022, S. Eskeland propose un système de chiffrement de diffusion à clé publique dont le but est de partager une clé privée d'un système symétrique à un réseau d'utilisateurs. Dans son système, le déchiffrement ne nécessite qu'une clé privée de taille fixe donc, il garantit une gestion aisée et une transmission rapide des clés. D'un autre côté, les utilisateurs resteront anonymes car le système n'a pas besoin d'une liste contenant les utilisateurs. L'efficacité semble bonne car le déchiffrement ne nécessite qu'une exponentiation modulaire. De plus, il est dit résistant aux attaques de collusion [1].

Notre travail était d'abord d'analyser la validité du système, de l'implémenter, d'analyser son efficacité puis de trouver une attaque pour montrer ses failles.

2 Le système d'Eskeland

Le système d'Eskeland, étudié dans ce rapport, est basé sur l'utilisation de sous-groupes RSA cachés. En d'autres termes, la sécurité du système repose sur le fait que chaque clé attribuée à chaque utilisateur est congrue à un même entier secret modulo un entier inconnu et dont on ne connaît pas la factorisation (on verra plus loin que cela n'empêche pas la collusion). La description du cryptosystème est la suivante :

2.1 Préliminaires et configuration

- Soit $n = pq$ le produit de deux grands nombres premiers secrets de la forme suivante :

$$p = 2p_0 \left(\prod_{j=1}^{\lceil N/2 \rceil} p_j \right) + 1 \quad \text{et} \quad q = 2q_0 \left(\prod_{j=\lceil N/2 \rceil + 1}^N p_j \right) + 1 \quad (1)$$

où N représente le nombre de destinataires. Les valeurs p_0 , q_0 , et p_j avec $1 \leq j \leq N$ sont des nombres premiers distincts et de grande taille, approximativement de même ordre de grandeur. Le niveau de sécurité λ est déterminé par $\lambda = \|p_j\|$ où $\|p_j\|$ désigne la longueur binaire de p_j (c'est-à-dire $2^{\|p_j\|-1} < p_j < 2^{\|p_j\|}$).

- Soit $g = \alpha^2 \bmod n$, où α est un générateur de $(\mathbb{Z}/p\mathbb{Z})^\times$ et de $(\mathbb{Z}/q\mathbb{Z})^\times$.
- Soit g_i pour $1 \leq i \leq N$, un générateur du sous-groupe G_i d'ordre $p_0 q_0 p_i$ de $(\mathbb{Z}/n\mathbb{Z})^\times$, défini par :

$$g_i = g^{\bar{p}_i} \bmod n \quad \text{où} \quad \bar{p}_i = \prod_{\substack{j=1 \\ j \neq i}}^N p_j$$

L'ordre de G_i est caché, car les paramètres p_0 , q_0 , et p_j sont secrets.

- On sélectionne un grand entier aléatoire secret γ , dont la taille en bits est au moins celle de n , et pour chaque utilisateur on définit :

$$y_i = g_i^\gamma \bmod n$$

- Les clés privées pour chaque utilisateur $P_i \in U$ sont calculées comme suit :

$$\gamma_i = \gamma \bmod p_0 q_0 \bar{p}_i$$

La configuration du système, appelée algorithme *Setup*, est définie comme suit : $(pk, sk) \leftarrow \text{Setup}(N, \lambda)$, où l'algorithme prend en entrée un paramètre de sécurité λ et le nombre d'utilisateurs N , et renvoie $pk = (\{g_i, y_i \mid 1 \leq i \leq N\}, n)$ et $sk = \{\gamma_i \mid 1 \leq i \leq N\}$.

2.2 Chiffrement

On sélectionne un ensemble de destinataires $T \subseteq U$ qui sont les cibles d'une diffusion sécurisée, où $R = U \setminus T$ désigne un ensemble d'utilisateurs révoqués. Puis on génère un entier secret aléatoire $r \in (\mathbb{Z}/n\mathbb{Z})^\times$ et on calcule l'entier suivant :

$$k_T = \left(\prod_{j \in R} y_j \right)^r \bmod n$$

Cet entier k_T correspond à une clé de chiffrement qui sera utilisée ultérieurement par un autre système symétrique. Dans la suite, nous ne considérons pas la partie du chiffrement symétrique et nous supposons que k_T est le texte clair (et si un attaquant est capable de le retrouver, il est bien capable de casser la confidentialité du système global). Le texte chiffré (également appelé en-tête de chiffrement, qui est ce que chaque utilisateur reçoit) est défini comme :

$$z = \left(\prod_{j \in R} g_j \right)^r \bmod n$$

Dans le cas particulier où $R = \emptyset$, alors $k_T = k_U = y_0^r$ et $z = g_0^r$.

2.3 Déchiffrement

À la réception de z , chaque utilisateur $P_i \in T$ est capable de reconstituer la clé de chiffrement k_T en effectuant une exponentiation modulaire :

$$k_T = z^{\gamma_i} \bmod n$$

On note qu'il n'y a qu'un seul élément de clé publique et un élément de clé privée (pour chaque utilisateur), et l'en-tête est le seul élément nécessaire pour le déchiffrement.

2.4 Validité

Montrons qu'on peut retrouver le message k_T à partir de (γ_i, z, n) quand l'utilisateur $P_i \in T$:

$$\begin{aligned}
k_{T,i} &\equiv z^{\gamma_i} \pmod{n} \\
&\equiv \left(\left(\prod_{k \in R} g_k \right)^r \right)^{\gamma_i} \pmod{n} \\
&\equiv \left(\prod_{k \in R} g_k^{r \cdot (\gamma \pmod{p_0 q_0 \bar{p}_i}) \pmod{p_0 q_0 p_k}} \right) \pmod{n}, \text{ car } g_k \text{ d'ordre } p_0 q_0 p_k \\
&\equiv \left(\prod_{k \in R} g_k^{r \cdot \gamma \pmod{p_0 q_0 p_k}} \right) \pmod{n} \\
&\equiv \left(\prod_{k \in R} g_k^{\gamma} \right)^r \pmod{n} \\
&\equiv \left(\prod_{k \in R} y_k \right)^r \pmod{n} = k_T
\end{aligned}$$

Ces congruences successives sont valides car l'ordre du sous-groupe G_k g n r  par g_k est $p_0 q_0 p_k$, et $p_0 q_0 p_k$ divise le module $p_0 q_0 \bar{p}_i$ de γ_i pour $i \neq k$ (qui n'est pas le cas quand l'utilisateur $P_i \in R$). Par cons quent, pour deux utilisateurs P_i et $P_j \in T$ poss dant deux cl s priv es distinctes (γ_i, γ_j) , ils calculeront la m me cl  k_T .

2.5 S lection des param tres

Il est sugg r  de choisir $\|p_0\| = \|q_0\| = 2\lambda$, et de mani re similaire $\|p_j\| = \lambda$, $1 \leq j \leq N$, afin d'assurer une distribution suffisamment large de sous-groupes et de cl s priv es.

λ	80	80	80	112	112	112	128	128	128
N	10	20	50	10	20	50	10	20	50
$\ n\ $	1120	1920	4320	2048	2688	6048	3072	3072	6912

TABLE 1 – Param tres de s curit  λ et longueur binaire de $n = pq$ en fonction du nombre d'utilisateurs N

3 L'impl mentation du syst me

La premi re  tape de notre d veloppement exp rimental a consist    mettre en  uvre le syst me d crit dans la section pr c dente pour mieux comprendre en d tail le comportement attendu et les algorithmes garantissant la fiabilit  des informations s curis es. La principale difficult  de l'impl mentation repose sur la mani re de manipuler les grands nombres entiers n cessaires   son fonctionnement. C'est pourquoi nous avons opt  pour le langage Python en raison de la facilit  pour manipuler les grands entiers et de la grande

disponibilité de bibliothèques supplémentaires dont nous parlerons plus loin. D'autres difficultés reposent sur la génération des nombres premiers (qui sont utilisés partout dans le système) et la création des générateurs d'anneaux du type $\mathbb{Z}/n\mathbb{Z}$.

Le problème de la génération des nombres premiers peut être résolu de plusieurs manières. Une méthode immédiate consiste à rechercher le nombre premier de la taille souhaitée dans une base de données contenant une certaine plage de nombres premiers. Cependant, nous avons opté pour une solution purement aléatoire sans avoir besoin de ressources externes : pour chaque nombre premier de taille binaire μ on tire un nombre aléatoire entre $2^{\mu-1}$ et 2^μ et on teste sa primalité, s'il est premier l'algorithme se termine, sinon un nouveau nombre est extrait.

Clarifions cette démarche. Le théorème des nombres premiers nous dit qu'asymptotiquement

$$\pi(t) \underset{t \rightarrow \infty}{\sim} t \ln(t)^{-1}$$

avec π fonction de comptage des nombres premiers (autrement dit la fonction qui pour toute valeur $t > 0$ nous donne la quantité de nombres premiers inférieurs à t) [6]. On en déduit que la densité des nombres premiers sur les nombres naturels approche $\ln^{-1}$. Il est alors garanti que le tirage aléatoire des nombres donne éventuellement un nombre premier dans un intervalle de temps raisonnable. De plus, pour tester leur primalité, nous utilisons l'algorithme de Miller-Rabin, qui est un test de primalité probabiliste qui, avec une certitude de $1/4$, nous dit si un nombre donné n est composé (donc, il est exécuté plusieurs fois pour garantir sa primalité ou sa non-primalité) [3] [8].

En outre, comme décrit ci-dessus, la création des clés nécessite des générateurs des anneaux $\mathbb{Z}/p\mathbb{Z}$ et $\mathbb{Z}/q\mathbb{Z}$. Ces générateurs sont obtenus par une recherche naïve parmi les éléments inversibles de $\mathbb{Z}/\phi(n)\mathbb{Z}$, où $\phi(n)$ représente la fonction totient d'Euler évaluée à n (on rappelle que $\phi(n)$ nous donne le nombre des entiers inférieurs à n qui sont premiers avec n , qui équivaut à la cardinalité de l'ensemble des nombres inversibles dans $\mathbb{Z}/n\mathbb{Z}$). Or, il nous faut notamment un générateur de $\mathbb{Z}/n\mathbb{Z}$. Ceci est obtenu en identifiant $\mathbb{Z}/n\mathbb{Z}$ avec $\mathbb{Z}/p\mathbb{Z} \times \mathbb{Z}/q\mathbb{Z}$ donné par l'isomorphisme d'anneaux du théorème chinois des restes.

Le théorème chinois des restes est un résultat mathématique fondamental qui permet de résoudre simultanément un système de congruences. Supposons que nous ayons les congruences suivantes :

$$x \equiv a_1 \pmod{n_1}$$

$$x \equiv a_2 \pmod{n_2}$$

où n_1 et n_2 sont des nombres premiers entre eux. Selon le théorème, il existe une unique solution x modulo $n_1 n_2$. De plus, si x_1 et x_2 sont deux solutions modulo $n_1 n_2$, alors elles sont congruentes modulo $n_1 n_2$. En particulier,

$$x = a_1 n_2 m_2 + a_2 n_1 m_1 \pmod{n_1 n_2}$$

avec (m_1, m_2) les coefficients de Bézout de l'équation $n_1 m_1 + n_2 m_2 = 1$.

Dans le contexte de notre étude, si α_p et α_q sont des générateurs de $\mathbb{Z}/p\mathbb{Z}$ et de $\mathbb{Z}/q\mathbb{Z}$ respectivement alors

$$\alpha = \alpha_p \cdot (q^{-1} \pmod{p}) \cdot q + \alpha_q \cdot (p^{-1} \pmod{q}) \cdot p$$

est l'image inverse de (α_p, α_q) par l'isomorphisme des restes chinois donc un générateur de $\mathbb{Z}/n\mathbb{Z}$ avec $(q^{-1} \bmod p)$ l'entier inverse de q modulo p et $(p^{-1} \bmod q)$ l'entier inverse de p modulo q .

Les autres opérations décrites dans le système d'Eskeland sont suffisamment élémentaires pour être mises en œuvre directement : par exemple l'exponentiation modulaire rapide, le calcul du pgcd, le calcul des coefficients de Bézout (nécessaire au théorème chinois des restes), etc. Cependant, nous avons choisi de les mettre en œuvre nous-mêmes pour approfondir notre compréhension du système. Toutes ces fonctions sont contenues dans le fichier `arithmetics.py` du répertoire GitHub de l'implémentation [7].

3.1 Résultats expérimentaux

Pour tester l'efficacité empirique du système proposé, nous nous sommes concentrés sur l'analyse des temps d'exécution du Setup (i.e. la génération des clés). Les résultats obtenus sont les suivants :

λ	80	80	80
N	10	20	50
μ	0,955	4,18	78,94
max	2,259	17,157	217,76
min	0,212	1,01	16,51
σ	0,515	3,21	45,57

λ	112	112	112
N	10	20	50
μ	2,898	17,403	293,65
max	6,089	44,59	883,42
min	0,687	2,998	57,53
σ	1,46	10,39	173,9

λ	128	128	128
N	10	20	50
μ	4,56	26,1	478,33
max	16,56	69,147	1194,5
min	0,531	8,966	100,58
σ	2,7	14,14	234,96

TABLE 2 – Tableaux des temps d'exécution (en secondes) selon les paramètres (λ, N) .
Processeur Intel I9-10900 ; 50 exécutions par paramètre

Nous nous sommes concentrés uniquement sur une analyse de la configuration car les algorithmes de chiffrement et de déchiffrement sont assez simples : ils n'ont besoin que d'une exponentiation modulaire et donc leurs temps d'exécution considérablement

inférieurs à ceux de la génération de clés. En effet, la configuration initiale nécessite non seulement la génération de $N + 2$ nombres premiers distincts mais aussi la génération de p et q (voir équation 1) qui dépendent eux mêmes des p_i et doivent aussi être premiers. Pourtant, rien ne nous assure que ce sera le cas donc la génération doit être relancée si les p et q obtenus sont composés.

D'après la Table 2, on constate que lorsque le paramètre de sécurité λ et le nombre d'utilisateurs N augmentent, le temps d'exécution du Setup augmente aussi de manière significative. Par exemple, pour un paramètre de sécurité de 80, le temps d'exécution moyen de la configuration initiale pour un réseau de 10, 20 et 50 utilisateurs varie entre 0,9 secondes, 4,1 secondes et 1 minute et 18 secondes. De même, à $N = 20$ fixé les temps d'exécution varient entre 4,1 secondes, 17,4 seconde et 26 secondes pour $\lambda = 80, 112$ et 128 respectivement.

Ces temps d'exécution variés mais importants sont dus à la densité des nombres premiers parmi les entiers dont on mentionne dans la section précédente. On rappelle que chaque nombre p_i appartient à l'intervalle $[2^{\lambda-1}, 2^\lambda]$ et $p_0, q_0 \in [2^{2\lambda-1}, 2^{2\lambda}]$. Donc, comme p et q sont de l'ordre $2^{(\frac{N}{2}+2)\lambda+1}$, il est peu probable qu'ils soient premiers à la première tentative. En effet, si on suppose que p (respectivement q) est uniformément choisi dans l'ensemble « $E = [2^{(\frac{N}{2}+2)(\lambda-1)+2}, 2^{(\frac{N}{2}+2)\lambda+1}] \setminus 2\mathbb{N}$ », alors :

$$\mathbb{P}(p \text{ premier}) \approx \frac{\pi(2^{(\frac{N}{2}+2)\lambda+1}) - \pi(2^{(\frac{N}{2}+2)(\lambda-1)+2})}{|E|}$$

Pratiquement, de façon exemplaire, lorsque $(\lambda, N) = (80, 20)$, la probabilité que p soit premier approche 0,3%, ce qui explique les différences de temps (pour une analyse plus précise de la complexité temporelle, voir l'annexe 7.1).

4 Les attaques

Contrairement aux prétentions de l'auteur, les attaques trouvées se basent sur le principe de collusion. Un ou plusieurs utilisateurs, rejetés ou non, peuvent collaborer avec leurs clés secrètes pour trouver soit la décomposition de $n = pq$, soit une décomposition de $p - 1$ et $q - 1$ qui revient à trouver la liste de p_0, q_0 et des p_i .

Dans un premier temps, on remarque que la différence entre deux clés γ_i et γ_j nous donne déjà beaucoup d'informations. En particulier, nous avons :

$$\begin{aligned} \gamma_i &= \gamma + kp_0q_0\bar{p}_i, k \in \mathbb{Z} \\ \gamma_j &= \gamma + k'p_0q_0\bar{p}_j, k' \in \mathbb{Z} \\ \implies \Delta_{i,j} &:= \gamma_i - \gamma_j = p_0q_0(p_jk - p_ik') \prod_{l=1, l \neq \{i,j\}}^N p_l \end{aligned} \tag{2}$$

Nous savons que tous les p_i ont à peu près la même taille binaire λ et que p_0 et q_0 sont de taille 2λ . De plus, comme décrit dans l'article d'Eskeland, λ vaut au plus 128. Par conséquent, nous pouvons appliquer en toute sécurité l'algorithme de factorisation ECM de la courbe elliptique de Lenstra qui dépend du plus petit facteur (donc environ 2^λ) pour

factoriser $\Delta_{i,j}$ en temps sous-exponentiel [5]. Appliqué à $\Delta_{i,j}$, l'ECM de Lenstra donnerait une liste contenant tous les facteurs premiers p_i (sauf p_i et p_j mais incluant p_0 et q_0) plus la factorisation de $(p_j k - p_i k')$. Néanmoins, nous sommes heuristiquement capables de filtrer le sous-ensemble des facteurs de $(p_j k - p_i k')$, puisque les éléments de ce sous-ensemble peuvent être de tailles binaires variées (en dehors de l'intervalle $[2^{\lambda-1}, 2^\lambda]$) par rapport à la taille des nombres premiers p_i qui sont uniformément choisis dans l'intervalle. Définissons ainsi cette liste de facteurs premiers que composent $\Delta_{i,j}$ excluant les valeurs inconnues p_i et p_j et les valeurs filtrées :

$$P_{i,j} := \{p_0, q_0\} \cup \{p_k, 1 \leq k \leq N\} \setminus \{p_i, p_j\} \quad (3)$$

4.1 Le cas de plusieurs clés secrètes

Étant donné au moins une autre clé γ_k , nous pouvons itérer le processus décrit dans l'équation 2 pour obtenir $\Delta_{i,j}$, $\Delta_{i,k}$ et $\Delta_{j,k}$. Ainsi, l'ensemble des facteurs premiers p_0 , q_0 et $\{p_l, 1 \leq l \leq N\}$ est obtenu par l'union $P_{i,j} \cup P_{i,k} \cup P_{j,k}$.

4.2 Le cas de deux clés secrètes

Si aucune autre clé n'est fournie en plus de γ_i et γ_j , il reste du travail à faire pour trouver les nombres premiers manquants p_i et p_j .

4.2.1 Factorisation par PGCD

Par symétrie des cas, dans cette section nous supposons que p_i et p_j sont des facteurs premiers de $p-1$. Si tel est le cas, nous pouvons appliquer l'algorithme suivant (Algorithm 1) pour factoriser $n = pq$.

Algorithm 1 n -factoring

Require: $P_{i,j}$, n

Ensure: q tel que $n = pq$

$q \leftarrow 1$

$e \leftarrow 1$

$a \bmod q$, un nombre choisi aléatoirement

for all $r \in P_{i,j}$ **do**

$e \leftarrow e \cdot r$

$s \leftarrow \text{pgcd}(a^e - 1, n)$

if $s \notin \{1, n\}$ **then**

$q \leftarrow s$

end if

end for

Soit $\omega := q - 1 = q_0 \prod_{l=\lceil \frac{N}{2} \rceil + 1}^N p_l$. Le petit théorème de Fermat nous dit que si q est un nombre premier alors

$$\forall a \in \mathbb{Z}/q\mathbb{Z}, a^\omega \equiv 1 \bmod q$$

En particulière, $a^\omega - 1$ est un multiple de q , et $\forall z \in \mathbb{N}$, $a^{z \cdot \omega} - 1$ multiple de q . Or, par définition de q (voir l'équation 1) et le fait que p_i et p_j sont des facteurs de $p - 1$, après un certain nombre d'étapes dans la boucle principale, on se retrouverait avec $z_0 \cdot \omega$ (un multiple de $q - 1 = \omega$) pour un certain $z_0 \in \mathbb{N}^*$. Finalement, une fois trouvée la valeur $z_0 \cdot \omega$, on factorise n en appliquant le pgcd, i.e. $\text{pgcd}(a^{z_0 \cdot \omega} - 1, n) = q$.

4.2.2 Fractions continues

De même si $p_i | p - 1$ et $p_j | q - 1$ (respectivement si $p_i | q - 1$ et $p_j | p - 1$) alors une attaque basée sur les fractions continues, inspirée de la cryptanalyse des exposants secrets courts de RSA [11], peut être implémentée pour trouver p_i et p_j . Cette cryptanalyse qui inspire notre attaque, utilise une méthode basée sur des fractions continues pour révéler la clé secrète d lorsque d est suffisamment petit ($d < \frac{1}{3}N^{0.25}$, N faisant partie de la clé publique RSA dans ce contexte).

On rappelle que les fractions continues sont une représentation alternative pour les nombres réels qui permet d'exprimer un nombre sous la forme d'une séquence finie ou infinie de fractions partielles. Le développement en fraction continue (DFC) d'un certain nombre $\alpha \in \mathbb{R}$ est généralement représentée comme une expression de la forme :

$$\alpha = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{a_3 + \frac{1}{\dots}}}}$$

où les a_i sont des entiers positifs. De plus, on définit :

$$p_k := a_k p_{k-1} + p_{k-2} \text{ et } q_k := a_k q_{k-1} + q_{k-2} \quad (4)$$

avec $(p_0, q_0) = (a_0, 1)$ et $(p_1, q_1) = (a_0 a_1 + 1, a_1)$. Chaque fraction partielle p_k/q_k est appelée la k -ième réduite du développement en fraction continue de α . Pour tout entier k , nous utilisons la notation $p_k/q_k = [a_0, a_1, \dots, a_k]$ pour désigner la k -ième réduite [10]. On remarque que le DFC d'un nombre $x \in \mathbb{R}$ est fini si et seulement si c'est un nombre rationnel.

Le théorème de Legendre sur les fractions continues nous dit que si $\alpha \in \mathbb{R}$ et $p/q \in \mathbb{Q}$ sont tels que

$$\left| \alpha - \frac{p}{q} \right| \leq \frac{1}{2q^2}$$

alors cela impose que p/q soit une approximation de α comme un développement en fraction continue tronquée de α . Autrement dit, p/q est l'une des réduites du développement en fraction continue de α .

Dans le cadre du projet, on remarque que $n = pq$ est assez proche de $\phi(n) = (p - 1)(q - 1)$ du fait de la grande taille des nombres premiers qui composent $p - 1$ et $q - 1$. Ainsi, ayant $P_{i,j}$ (défini dans l'équation 3) on pose

$$p'q' := \prod_{p_k \in P_{i,j}} p_k$$

indiquant la multiplication de tous les nombres premiers connus du système sauf p_i et p_j . On remarque, après une vérification immédiate (par un calcul simple explicite), que pour tous les paramètres (N, λ) dans le tableau 1, nous avons :

$$\left| \frac{p'q'}{n} - \frac{p'q'}{\phi(n)} \right| = \left| \frac{p'q'}{n} - \frac{1}{4p_i p_j} \right| \leq \frac{1}{2 \cdot (4p_i p_j)^2} \quad (5)$$

considérant que chaque $\|p_k\| = \lambda$, $\|p_0\| = \|q_0\| = 2\lambda$ et $\|\phi(n)\| = (N + 4)\lambda$ (pour une démonstration plus précise, voir l'annexe 7.3). Nous concluons que $1/(4p_i p_j)$ est une approximation de $p'q'/n$ dont le développement en fraction continue est calculable à l'aide de l'algorithme d'Euclide.

Comme on voit dans l'Algorithme 2, une fois obtenue la séquence $[0, a_1, \dots, a_m]$ représentant le DFC de $p'q'/n$, on peut tester pour tout $\ell \in \{1, \dots, m\}$ si la ℓ -ième réduite $\beta_\ell = [0, a_1, \dots, a_\ell]$ est bien égal à $1/4p_i p_j$ l'inverse du produit des nombres cherchés. Le théorème d'Euler nous dit que $\forall a \in (\mathbb{Z}/n\mathbb{Z})^\times$, $a^{\phi(n)} \equiv 1 \pmod n$. Alors, pour déterminer si le nombre β_ℓ est bien celui qu'on cherche, on peut tester pour $A \in (\mathbb{Z}/n\mathbb{Z})^\times$, si

$$\forall a \in A, a^{\beta_\ell^{-1} p'q'} \equiv 1 \pmod n$$

qui est vrai si $\beta_\ell = (4p_i p_j)^{-1}$ par le théorème d'Euler, étant donné que

$$4p_i p_j p'q' = (p-1)(q-1) = \phi(n)$$

.

Algorithm 2 Inverse approximation

Require: $n, p'q', M$ (le nombre de tests qu'on fera pour déterminer si on a bien trouvé notre résultat)

Ensure: t tel que $4p_i p_j p'q' = \phi(n)$

$L \leftarrow DFC(p'q'/n)$, la liste $[0; a_1 \dots a_m]$

$t \leftarrow -1$

for $\ell \leftarrow 1$ à $|L|$ **do**

$A \leftarrow L[0 : \ell]$

▷ La liste correspondant à la ℓ -ième réduite

$\beta \leftarrow Evaluate(A)$

▷ La valeur fractionnelle de la ℓ -ième réduite

$echec \leftarrow false$

for $m \leftarrow 1$ à M **do**

$a \leftarrow \text{randomInteger}(2, n-1)$

if $a^{\beta^{-1} p'q'} \not\equiv 1 \pmod n \wedge \text{pgcd}(a, n) = 1$ **then**

$echec \leftarrow true$

end if

end for

if $\neg echec$ **then**

$t \leftarrow \beta^{-1}$

end if

end for

λ	80	80	80	112	112	112	128	128	128
N	10	20	50	10	20	50	10	20	50
μ	0,074	0,31	3,03	0,18	0,79	7,74	0,25	1,15	11,50
σ	0,001	0,01	0,08	0,004	0,01	0,1	0,004	0,02	0,15

TABLE 3 – Tableau des temps d'exécution (en secondes) de l'attaque par fraction continue selon les paramètres (λ, N) . Processeur Intel I9-10900 ; 50 exécutions par paramètre ; Taux de succès = 100%

De même que pour les temps d'exécution du Setup, on constate que lorsque le paramètre de sécurité λ et le nombre d'utilisateurs N augmentent, le temps d'exécution de l'attaque augmente aussi de manière significative (voir Table 3). Cependant, il est aisé de voir que la complexité de l'attaque est polynomiale en le nombre d'utilisateurs N et en le paramètre de sécurité λ (alors qu'en cryptographie, il est attendu que la meilleure attaque possible ait une complexité de l'ordre de $\Omega(2^\lambda)$).

Dans le cadre d'étude de l'algorithme 2, on suppose que le nombre de tests M est fixé. De plus, on considère que les opérations de multiplication sont faites en $O(\lambda N \cdot \log(\lambda N))$, utilisant l'algorithme de Harvey-Hoeven [4], et que l'exponentiation modulaire est en $O((\lambda N)^2 \log(\lambda N))$ utilisant la méthode exponentiation par élévation au carré à l'aide de la multiplication Harvey-Hoeven [2]. Sans utiliser de méthode de multiplication rapide, les complexités respectives de ces deux opérations sont $O((\lambda N)^2)$ et $O((\lambda N)^3)$. En outre, on constate que le calcul du DFC (voir équation 4) est fait par l'algorithme d'Euclide du pgcd, donc on obtient une liste L de $O(\lambda N)$ éléments (on rappelle que le pgcd de deux nombres a et b est en $O(\min(\log(a), \log(b)))$) ce qui implique $O(\lambda N)$ tours de boucle. On remarque que tout comme l'exponentiation rapide, le calcul du $\beta = \text{Evaluate}(A)$ est en $O((\lambda N)^2 \log(\lambda N))$. Ce calcul est fait en parcourant une liste de ℓ éléments faisant une multiplication à chaque tour de boucle. Donc, on en déduit que l'algorithme 2 est en $O((\lambda N)^3 \log(\lambda N))$ qui est une complexité polynomiale (mais qui peut être cependant un peu coûteuse pour de grands réseaux d'utilisateurs). Notons que pour les tailles de paramètre suggérées par Eskeland, il est totalement impossible à l'heure actuelle de factoriser un entier RSA n de plus de 1 000 bits en un temps pratique (sans utiliser la structure additionnelle et donc la faiblesse que nous avons trouvée).

5 Conclusion

Après avoir implémenté et analysé le schéma de chiffrement d'Eskeland, nous concluons que le système n'est pas suffisamment fiable pour être mis en œuvre dans un contexte réel. Comme vu dans la section 4 sur les attaques, il existe plusieurs manières de compromettre le réseau des utilisateurs. Même avec seulement deux collaborateurs, nous pouvons trouver la liste des facteurs des sous-groupes cachés qui composent $(p-1)(q-1)$. Avec ces facteurs, il est assez élémentaire de dupliquer les clés de n'importe quel autre utilisateur sur le réseau : on peut simplement résoudre le système suivante pour récupérer la valeur de γ ,

$$\gamma \equiv \begin{cases} \gamma_i \bmod p_0 q_0 \bar{p}_i \\ \gamma_j \bmod p_i \end{cases}$$

Même si ces failles de sécurité sont résolues en modifiant la définition des clés privées, le système n'est pas évolutif. En effet, comme on le voit dans la section des résultats expérimentaux, la configuration initiale des nombres premiers est surlinéaire en fonction du niveau de sécurité souhaité et du nombre de destinataires. Certainement, même avec des serveurs avec des grands puissances de calcul, on peut s'attendre à ce que dans les grands réseaux d'utilisateurs les temps d'exécution soient considérables pouvant potentiellement prendre plusieurs heures. À titre d'exemple, avec nos ressources on s'attendrait à ce qu'une exécution sur un réseau de 10 000 utilisateurs avec un paramètre de sécurité $\lambda = 128$ prenne plus de 26 heures. On remarque aussi que non seulement ces temps augmentent exceptionnellement mais aussi la taille des clés (voir Table 1). Par exemple, si on veut partager un message dans un réseau d'uniquement 1 000 destinataires avec un niveau de sécurité minimal (soit $\lambda = 80$), il faudrait générer un module n d'une taille d'environ 80ko. On conclut donc que ce système ne peut pas être mis en œuvre à grande échelle.

6 Références

- [1] Eskeland, S. (2022). *Collusion-resistant broadcast encryption based on hidden RSA subgroups*. Cryptology ePrint Archive.
- [2] Gueron, S. (2012). *Efficient software implementations of modular exponentiation*. Journal of Cryptographic Engineering. 2 (1) : 31–43.
- [3] Harari, S. (1983). *Primality Testing—A Deterministic Algorithm*. Secure Digital Communications, 121-125.
- [4] Harvey, D. ; Van Der Hoeven, J. (2021). *Integer multiplication in time $O(n \log n)$* . Annals of Mathematics. 193 (2) : 563–617.
- [5] Lenstra, A. K., Hendrik Jr, W. (1993). *The development of the number field sieve* (Vol. 1554). Springer Science and Business Media.
- [6] Jameson, G. J. O. (2003). *The prime number theorem* (No. 53). Cambridge University Press.
- [7] Pulido-Cornejo, D., Huang K., Douar, L. (2023). *2IN013_RSA-based-broadcast-encryption*. GitHub. https://github.com/puli-101/2IN013_RSA-based-broadcast-encryption
- [8] Rabin, M. O. (1980), *Probabilistic algorithm for testing primality*, Journal of Number Theory, 12 (1) : 128–138, doi :10.1016/0022-314X(80)90084-0
- [9] Smart, P. N. (2016). *Cryptography made simple*. Springer International Publishing Switzerland.
- [10] Vergnaud, D. (2018). *Exercices et problèmes de cryptographie - 3e édition*. Dunod 265-269.
- [11] Wiener, M. J. (1990). *Cryptanalysis of short RSA secret exponents*. IEEE Transactions on Information theory, 36(3), 553-558.

7 Annexes

7.1 Analyse théorique de la génération des nombres premiers

La génération d'un nombre premier aléatoire (algorithme 3) se fait en $O(\lambda^4)$. Si l'on considère X la variable aléatoire discrète qui nous indique le nombre de tentatives avant le premier tirage réussi d'un nombre premier dans l'ensemble $\llbracket 2^\lambda, 2^{\lambda-1} \rrbracket$; alors asymptotiquement en utilisant l'approximation de la fonction de comptage des nombres premiers et la densité donnée dans la section 3, on en déduit que $\mathbb{E}[X] = O(\lambda)$ [6]. En d'autres termes, nous attendons $O(\lambda)$ boucles avant de trouver notre premier nombre premier. En outre, le test de primalité de Miller Rabin, qui se fait à chaque tour de boucle, est en $O(\lambda^3)$ [8]. On en déduit donc une complexité totale de $O(\lambda^4)$.

Algorithm 3 genPrime

Require: λ

Ensure: p_i tel que $\|p_i\| = \lambda$ et p_i premier

```
 $p_i \leftarrow 1$   
 $isPrime \leftarrow false$   
while  $\neg isPrime$  do  
   $p_i \leftarrow randInteger(2^{\lambda-1}, 2^\lambda)$   
   $isPrime \leftarrow millerRabin(p_i)$   
end while
```

De même, à λ fixé, lorsque le nombre d'utilisateurs N augmente, l'algorithme 4, pour générer le nombre p tel que spécifié dans l'équation 1 est en $O(N^4)$. En effet, de la même façon que pour la génération des nombres premiers de taille λ , on attend $O(\|p\|) = O(N\lambda) = O(N)$ itérations avant le premier succès. Aussi dans le FOR de la boucle principale, nous itérons $O(N)$ fois. À chaque itération, on tire un nombre premier de taille λ qui est en $O(\lambda^4) = O(1)$, alors une complexité partielle de cette boucle de $O(N)$. Or, l'opération la plus lourde est de tester la primalité de p qui est à $O(\lambda^3 N^3) = O(N^3)$. On en déduit donc une complexité de $O(N^4)$.

Algorithm 4 Generate p

Require: N, λ

Ensure: $p = 2p_0 \left(\prod_{j=1}^{N/2} p_j \right) + 1$ premier

```
 $p \leftarrow 1$   
 $isPrime \leftarrow false$   
while  $\neg isPrime$  do  
   $p \leftarrow 2 \cdot genPrime(2\lambda)$   
  for  $i \leftarrow 1$  à  $\frac{N}{2}$  do  
     $p \leftarrow p \cdot genPrime(\lambda)$   
  end for  
   $p \leftarrow p + 1$   
   $isPrime \leftarrow millerRabin(p)$   
end while
```

7.2 Exemple d'une fraction continue

Prenons la fraction continue suivante :

$$1 + \frac{1}{2 + \frac{1}{2 + \frac{1}{2 + \frac{1}{2 + \dots}}}}$$

Cette fraction continue représente la racine carrée de 2. En la simplifiant, nous pouvons obtenir des approximations successives de $\sqrt{2}$ en tronquant la séquence à un certain nombre de termes. Par exemple, en utilisant les 5 premiers termes de la fraction continue, nous avons :

$$\sqrt{2} \approx 1 + \frac{1}{2 + \frac{1}{2 + \frac{1}{2 + \frac{1}{2}}}} = \frac{17}{12} \approx 1.41666$$

Plus nous ajoutons de termes à la fraction continue, plus notre approximation de $\sqrt{2}$ devient précise.

7.3 Démonstration de l'application du théorème de Legendre

Définissons, par une majoration grossière, les valeurs suivantes à partir des paramètres (N, λ) , considérant que chaque $p_k \sim 2^\lambda$ et $p_0 \sim q_0 \sim 2^{2\lambda}$.

$$\begin{aligned} n &= (2^{(\frac{N}{2}+2)\lambda+1} + 1)^2 \\ p'q' &= 2^{(N+2)\lambda} \\ \phi(n) &= 2^{(N+4)\lambda+2} \\ 4p_i p_j &= 2^{2+2\lambda} \\ a &:= 2^{(\frac{N}{2}+2)\lambda+1} \end{aligned}$$

On veut pouvoir déterminer si $1/(4p_i p_j)$ (l'inverse de nos valeurs inconnues) est une approximation du type fraction continue de $p'q'/n$. Pour cela on veut appliquer le théorème de Legendre. Or, pour appliquer le théorème de Legendre, il faut montrer l'inégalité suivante :

$$\left| \frac{p'q'}{n} - \frac{p'q'}{\phi(n)} \right| = \left| \frac{p'q'}{n} - \frac{1}{4p_i p_j} \right| = \frac{n - \phi(n)}{4p_i p_j n} \leq \frac{1}{2(4p_i p_j)^2}$$

Ceci équivaut à montrer que

$$8p_i p_j (n - \phi(n)) \leq n$$

Commençons par remplacer les variables par leur définition

$$\begin{aligned} 2^{3+2\lambda}((2^{(\frac{N}{2}+2)\lambda+1} + 1)^2 - 2^{(N+4)\lambda+2}) &\leq (2^{(\frac{N}{2}+2)\lambda+1} + 1)^2 \Leftrightarrow \\ 2^{3+2\lambda}(2 \times 2^{(\frac{N}{2}+2)\lambda+1} + 1) &\leq (2^{(\frac{N}{2}+2)\lambda+1} + 1)^2 \Leftrightarrow \\ (2^{3+2\lambda} - 1)(2a + 1) &\leq a^2 \Leftrightarrow \\ f(a) = a^2 - 2(2^{3+2\lambda} - 1)a - (2^{3+2\lambda} - 1) &\geq 0 \end{aligned}$$

Cherchons les racines (r_1, r_2) de

$$f : \mathbb{R} \rightarrow \mathbb{R} \text{ telle que } x \mapsto x^2 - 2(2^{3+2\lambda} - 1)x - (2^{3+2\lambda} - 1)$$

On constate que $f(a) \geq 0$ ssi $a \notin [r_1, r_2]$. Soient

$$\begin{aligned} \Delta &:= 4(2^{3+2\lambda} - 1)^2 + 4(2^{3+2\lambda} - 1) \\ r &:= (2^{3+2\lambda} - 1) \pm \sqrt{\Delta}/2 = 2^{3+2\lambda} - 1 \pm \sqrt{(2^{3+2\lambda} - 1)^2 + (2^{3+2\lambda} - 1)} \\ r_1 &:= 2^{3+2\lambda} - 1 - \sqrt{(2^{3+2\lambda} - 1)^2 + (2^{3+2\lambda} - 1)} \\ r_2 &:= 2^{3+2\lambda} - 1 + \sqrt{(2^{3+2\lambda} - 1)^2 + (2^{3+2\lambda} - 1)} = 2^{3+2\lambda} - 1 + \sqrt{(2^{3+2\lambda} - 1 + 1/2)^2 - 1/4} \end{aligned}$$

On remarque que $r_1 < 0$ car $2^{3+2\lambda} - 1 > 0$ alors nous sommes intéressés uniquement par r_2 . Or, le nombre $a \in \mathbb{N}$ donc on considère la partie entière de r_2

$$\lfloor r_2 \rfloor = \lfloor 2^{3+2\lambda} - 1 + (2^{3+2\lambda} - 1/2) \rfloor \sim 2^{4+2\lambda} \text{ pour } \lambda \text{ assez grand}$$

Alors $a > \lfloor r_2 \rfloor \Leftrightarrow (\frac{N}{2} + 2)\lambda + 1 > 4 + 2\lambda \Leftrightarrow \frac{N}{2}\lambda > 3 \Leftrightarrow N\lambda > 6$ qui est vrai pour l'ensemble des paramètres. Donc le théorème de Legendre peut être appliqué.